

Probeklausur Java 2: Weltraummission

Willkommen zur Probeklausur! In dieser Übung verwalten wir die Missionen, Astronauten und Raumschiffe der "Galactic Space Agency" (GSA). Die Klausur deckt die Themen **Optionals, Lambdas und die Stream API** ab.

Aufgabe 1 - Optional (Astronauten & Raumanzüge)

Implementiere die Klassen `SpaceSuit`, `Astronaut` und `Task01`.

Klassendiagramm / Struktur

```
public record SpaceSuit(String model, double oxygenLevel) {}
```

Hinweise zur Klasse `Astronaut`

- Die Klasse hat die Attribute `String name` und `Optional<SpaceSuit> suit`.
- Die Konstruktoren sollen alle Attribute initialisieren. Stelle sicher, dass kein Fehler ausgelöst wird, falls eine `null`-Referenz für den Raumanzug übergeben wird. Es soll zwei Konstruktoren geben: einen nur mit Namen und einen mit Name und Raumanzug.
- Die Methode `toString` soll das Objekt als `String` zurückgeben. Berücksichtige nur Attribute mit Werten. Haben beide Attribute einen Wert (Name und Modell des Anzugs), trenne sie durch ein Leerzeichen.
- Die Methode `isReadyForSpacewalk` soll prüfen, ob der Astronaut einen Anzug trägt und dessen `oxygenLevel` größer als 50.0 ist.

Hinweise zur Klasse `Task01`

Erstelle in der `main`-Methode einen Astronauten namens "Buzz" mit einem "EVA-Mark3" Raumanzug (100.0 Sauerstoff). Gib den Astronauten auf der Konsole aus. Entferne anschließend seinen Raumanzug (setze ihn auf ein leeres `Optional`) und prüfe mit `isReadyForSpacewalk`. Falls er noch bereit wäre (was nicht der Fall sein sollte), gib eine Warnung aus.

Aufgabe 2 - Lambdafunktionen (Missionen)

Implementiere die Klassen `Mission`, `MissionType` (Enum) und `Task02`.

Klassendiagramm / Struktur

```
import java.util.List;

public enum MissionType {
    EXPLORATION, RESUPPLY, MAINTENANCE, COLONIZATION;

    public static List<MissionType> getDangerousTypes() {
        return List.of(MissionType.EXPLORATION, MissionType.MAINTENANCE);
    }
}
```

Hinweise zur Klasse `Mission` (Record)

Der Record `Mission` hat die Attribute `String name`, `MissionType type`, und `double budget`.

- Das Attribut `isDangerous` (als statisches Feld) soll eine Lambdafunktion (Predicate) enthalten, die ermittelt, ob die `Mission` gefährlich ist (nutze die Methode aus dem Enum).
- Das Attribut `cutBudget` soll eine Lambdafunktion (Function) enthalten, die eine bestehende `Mission` in eine angepasste Version umwandelt. Der Titel soll den Suffix " (Budget Cut)" erhalten und das Budget soll um 20% reduziert werden ($\ast 0.8$).
- Die Methode `isMoreExpensiveThan(double amount)` soll eine Lambdafunktion (Predicate) zurückgeben, die prüft, ob eine `Mission` mehr Budget hat als der Parameterwert.
- Die Methode `getMissions()` soll einen Stream von einer einzigen frei gewählten `Mission` zurückgeben (z.B. "Apollo 20", EXPLORATION, 500.0).

Hinweise zur Klasse `Task02`

Verwende für die nachfolgende Abfolge die statischen Methoden/Felder der Klasse `Mission`. Erzeuge einen Stream von Missionen, kürze deren Budget, filtere so, dass nur noch gefährliche Missionen übrig bleiben, und filtere danach nach Missionen, die immer noch teurer als 300.0 sind.

Aufgabe 3 - Streams (Space Agency Management)

Implementiere die Klassen `CrewMember`, `SpaceShip` und `SpaceAgency`.

Struktur

Die Records für Crew, Raumschiffe und Agency sind wie folgt aufgebaut (weitere Attribute bitte den Hinweisen entnehmen):

```
public record CrewMember(String name, double salary, List<String> skills) {}  
public record SpaceShip(String name, int launchYear, Map<String, Integer> cargo) {}
```

Hinweise zur Klasse `SpaceAgency`

Die `SpaceAgency` (Record) enthält `List<SpaceShip> ships` und `List<CrewMember> crew`. Setze die Anforderungen der Agency-Direktorin mit der Java Stream API um:

- **q1():** Die Direktorin möchte das Durchschnittsgehalt (double) der Crew-Mitglieder wissen, die "Piloting" oder "Navigation" als Skill haben.
- **q2():** Sie möchte die zwei "Mechanic"-Crewmitglieder (Skill "Mechanic") ermitteln, die das geringste Gehalt haben (aufsteigend sortieren). Gib für diese Personen einen Gruß auf der Konsole aus: "Gute Arbeit <Name>! Hier ist ein Bonus."
- **q3():** Gruppieren Sie alle Crewmitglieder, die den Skill "Leadership" **nicht** haben, nach dem ersten Buchstaben ihres Namens. Die Methode liefert eine `Map<Character, List<CrewMember>>`.
- **q4(String skill):** Die Direktorin sucht das "kosteneffizienteste" Crewmitglied für einen bestimmten Skill. Filtere nach dem Skill und finde das Mitglied mit dem niedrigsten (Minimum) Salary-to-Skill Ratio (Gehalt / Anzahl der Skills). Gebe ein `Optional<CrewMember>` zurück.
- **q5():** Liefere eine Liste (`List<String>`) aller großgeschriebenen Raumschiff-Namen, die mindestens ein Cargo-Element geladen haben, welches schwerer als 1000 kg ist (Werte in der Map). Keine Duplikate.